

Практическая теория типов

Лекция 10: От MLTT к Coq — CIC, Identity-типы и импредикативность

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты
- 3 Identity-типы и равенство
- 4 Продвинутое темы: аксиомы и альтернативы
- 5 CIC: Prop, Set, Type и импредикативность
- 6 Зависимые типы на практике
- 7 Заключение

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты
- 3 Identity-типы и равенство
- 4 Продвинутое темы: аксиомы и альтернативы
- 5 CIC: Prop, Set, Type и импредикативность
- 6 Зависимые типы на практике
- 7 Заключение

Большая картинка: от λ С к реальным системам

Определение (Martin-Löf Type Theory)

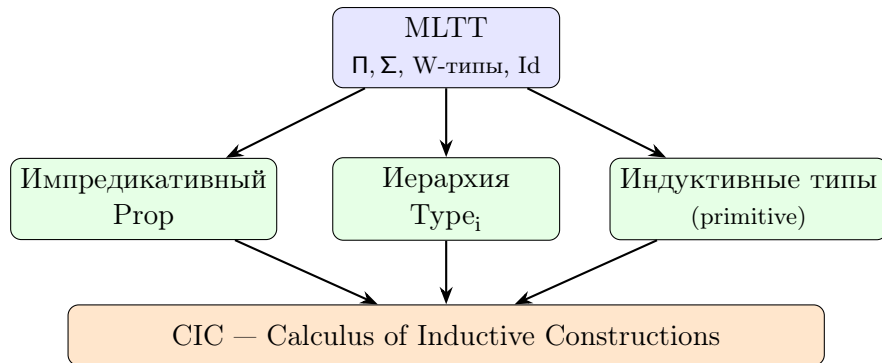
MLTT — теория зависимых типов: (1) $\prod_{x:A} B(x)$, $\sum_{x:A} B(x)$; (2) индуктивные типы; (3) $\mathcal{U}_0 : \mathcal{U}_1 : \dots$; (4) $\text{Id}_A(a, b) + J$.

Calculus of Inductive Constructions (CIC)

Coq реализует CIC — систему, родственную MLTT и CoC (Calculus of Constructions):

- Импредикативный Prop Иерархия $\text{Type}_0, \text{Type}_1, \dots$
- Identity-типы, equality, UIP vs HoTT

Что добавляет Coq к MLTT



- Импредикативный Prop: квантификация по всем Prop внутри Prop
- Иерархия Type_i: предикативные вселенные, избегающие парадокса Жирара
- Индуктивы как primitive: не только W-типы, но с positivity check

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты**
- 3 Identity-типы и равенство
- 4 Продвинутое темы: аксиомы и альтернативы
- 5 CIC: Prop, Set, Type и импредикативность
- 6 Зависимые типы на практике
- 7 Заключение

Из лекции 9 (один слайд)

- П-тип: $\Pi x : A. B(x)$ — зависимые функции / $\forall$ в логике
Пример: $\Pi n : \mathbb{N}. \text{Vec } A \ n \rightarrow A$ — head для вектора любой длины
- Σ -тип: $\Sigma x : A. B(x)$ — зависимая пара / $\exists$
Пример: $\Sigma n : \mathbb{N}. \text{Vec } A \ n$ — “список как вектор неизвестной длины”
- W-типы: универсальные индуктивные типы
Списки, деревья, натуральные числа — частные случаи W-типов

Важно: всё это уже “программируемая логика”, где типы — утверждения, а программы — доказательства.

Суждения в MLTT

Определение (Формы суждений MLTT)

- ❶ $\Gamma \vdash A : \mathcal{U}_i$ — A является типом уровня i
- ❷ $\Gamma \vdash a : A$ — терм a имеет тип A (typing judgement)
- ❸ $\Gamma \vdash a \equiv b : A$ — дефиниторное (judgemental) равенство
- ❹ $\Gamma \vdash A \equiv B : \mathcal{U}_i$ — равенство типов

где $\Gamma = x_1 : A_1, \dots, x_n : A_n$ — контекст.

Базовые типы MLTT

0 (пустой), 1 (единичный), Bool, $\mathbb{N}$, $\Pi x:A. B(x)$ (зав. функции), $\Sigma x:A. B(x)$ (зав. пары), $\text{Id}_A(a, b)$ (identity-тип — о нём далее!)

Варианты MLTT: интенциональная vs экстенциональная

Интенциональная MLTT (ITT)

- Identity-тип $\text{Id}_A(a, b)$ — тип
- $a \equiv b$ (дефиниторно) $\neq a = b$ (пропозиц.)
- UIP не выводим
- Разрешимая проверка типов

Примеры: Coq, Agda, Lean

Экстенциональная MLTT (ETT)

- Reflection rule: из $p : a = b$ следует $a \equiv b$
- Пропозиц. = дефиниторное
- UIP выводим автоматически
- Проверка типов неразрешима

Пример: NuPRL

Observational TT (OTT)

Компромисс: экстенциональное равенство без equality reflection, типизация разрешима.

Coq — интенциональная система

Варианты MLTT: иерархия вселенных

Проблема: $\text{Type} : \text{Type}$?

Наивное $\text{Type} : \text{Type}$ ведёт к парадоксу Жирара (аналог парадокса Рассела).

Решения:

- 1 Стратифицированные вселенные (Russell-style):

$\text{Type}_0 : \text{Type}_1 : \text{Type}_2 : \dots$

Кумулятивность: $A : \text{Type}_i \Rightarrow A : \text{Type}_{i+1}$

Примеры: Coq, Lean, Agda (с опцией)

- 2 Тарский-style вселенные:

$\mathcal{U}_i : \text{Type}$ — код вселенной, $\text{El} : \mathcal{U}_i \rightarrow \text{Type}$ — декодирование

Примеры: некоторые формализации MLTT

- 3 Полиморфизм вселенных (universe polymorphism):

Определения параметризованы уровнем: $\text{id} : \forall \ell. \forall A : \text{Type}_\ell. A \rightarrow A$

Варианты MLTT: индуктивные типы

Определение (W-типы)

Formation: $\frac{A : \mathcal{U} \quad B : A \rightarrow \mathcal{U}}{W_{x:A} B(x) : \mathcal{U}}$

Intro: $\text{sup} : \prod_{a:A} (B(a) \rightarrow W) \rightarrow W$

Минималистичный, но сложный для практики.

Схемы индуктивных типов (CIC)

Примитивные индуктивные определения:

- Inductive с конструкторами
- Positivity check
- Автогенерация элиминаторов
- Удобно для пользователя

Примеры: Coq, Lean, Agda

Расширения

- Индуктивно-рекурсивные типы (Agda): тип и функция определяются взаимно
- Higher Inductive Types (HoTT): конструкторы путей

Система	ИТТ/ЕТТ	Вселенные	UIP	Тактики
Coq	ИТТ	кумулятивные	опц.	Ltac, SSR
Agda	ИТТ	poly	опц.	нет*
Lean 4	ИТТ	кумулятивные	опц.	metaprog
Idris 2	ИТТ	кумулятивные	да	elaborator
NuPRL	ЕТТ	—	да	тактики
Cubical Agda	ИТТ+UA	poly	нет	нет

* Agda использует интерактивное построение термов, не тактический стиль.

UA = унивалентность, опц. = можно добавить аксиому K.

Computation rules: β и η для разных типов

Π -тип (зависимые функции)

- β -правило: $(\lambda x. t)(s) \equiv t[s/x]$
- η -правило: $f \equiv \lambda x. f(x)$ (если $x \notin FV(f)$)

Σ -тип (зависимые пары)

- β -правила: $\pi_1(a, b) \equiv a$, $\pi_2(a, b) \equiv b$
- η -правило: $p \equiv (\pi_1(p), \pi_2(p))$

Identity-тип

- β -правило для J : $J(x, P, d, x, \text{refl}_x) \equiv d$
- η -правило? Нет стандартного! Это связано с UIP/K.

Замечание: η -правила часто делают “пропозициональными”, не дефиниторными.

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты
- 3 Identity-типы и равенство**
- 4 Продвинутое темы: аксиомы и альтернативы
- 5 CIC: Prop, Set, Type и импредикативность
- 6 Зависимые типы на практике
- 7 Заключение

Две концепции равенства

Дефиниторное (judgemental)

$$\Gamma \vdash a \equiv b : A$$

Сводится $\beta/\delta/\iota$ -редукцией.

Пример:

$$(\lambda x. x) 3 \equiv 3$$

Coq проверяет это автоматически.

Нельзя “доказывать” — либо верно, либо нет.

Пропозициональное (identity-тип)

$$\text{Id}_A(a, b) \quad \text{или} \quad a =_A b$$

Это тип, который может быть обитаем или пуст.

В Coq:

```
Inductive eq (A:Type) (x:A)
  : A → Prop :=
| eq_refl : eq A x x.
(* Notation: x = y *)
```

Можно доказывать, использовать в рассуждениях.

Конструктор refl и его роль

Определение (Identity-тип: правила)

Formation:
$$\frac{\Gamma \vdash A : \mathcal{U} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash \text{Id}_A(a, b) : \mathcal{U}}$$

Introduction:
$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{refl}_a : \text{Id}_A(a, a)}$$

Семантика

- refl_a свидетельствует, что a равно самому себе
- Чтобы доказать $a = b$, нужно предъявить терм типа $\text{Id}_A(a, b)$

Пример

$\text{refl}_3 : \text{Id}_{\mathbb{N}}(1 + 2, 3)$ — работает, потому что $1 + 2 \equiv 3$ дефиниторно.

Элиминатор identity-типа: J

Определение (Элиминатор J: правило elimination)

$$\frac{\begin{array}{c} \Gamma \vdash A : \mathcal{U} \quad \Gamma \vdash x : A \\ \Gamma, y : A, p : \text{Id}_A(x, y) \vdash P(y, p) : \mathcal{U} \quad \Gamma \vdash d : P(x, \text{refl}_x) \\ \Gamma \vdash a : A \quad \Gamma \vdash e : \text{Id}_A(x, a) \end{array}}{\Gamma \vdash J(P, d, a, e) : P(a, e)}$$

Computation: $J(P, d, x, \text{refl}_x) \equiv d$

Интуиция:

- “Любой путь ведёт себя как refl ”
- Индукция по структуре пути

Вычислительное правило:

$$J(x, P, d, x, \text{refl}_x) \equiv d$$

При применении к refl возвращается базовый случай.

Замечание

Из J выводятся все стандартные свойства равенства: симметрия, транзитивность, конгруэнтность.

Определение (Transport)

$\text{transport} : \prod_{P:A \rightarrow \mathcal{U}} \prod_{x,y:A} \text{Id}_A(x,y) \rightarrow P(x) \rightarrow P(y)$

Def:

$\text{transport}(P, x, y, p, u) := J(\lambda y p. P(y), u, y, p)$

Определение (Congruence (ap))

$\text{ap} : \prod_{f:A \rightarrow B} \prod_{x,y:A} \text{Id}_A(x,y) \rightarrow \text{Id}_B(f(x), f(y))$

Def:

$\text{ap}(f, x, y, p) := J(\lambda y p. f(x) = f(y), \text{refl}_{f(x)}, y, p)$

Применение: переписывание под индексами

Если $v : \text{Vec } A \ n$ и $p : n = m$, то $\text{transport}_{\text{Vec } A}(p, v) : \text{Vec } A \ m$.

Симметрия из J: подробный вывод

Цель

Построить $\text{sym} : \prod_{x,y:A} (x =_A y) \rightarrow (y =_A x)$

Применение J

Напомним тип J:

$$J : \prod_{x:A} \prod_{P:\prod_{y:A} (x=y) \rightarrow \text{Type}} P(x, \text{refl}_x) \rightarrow \prod_{y:A} \prod_{p:x=y} P(y, p)$$

Выбираем мотив: $P(y, p) := (y =_A x)$

Базовый случай: нужен терм типа $P(x, \text{refl}_x) = (x =_A x)$

Подходит: $\text{refl}_x : x =_A x \checkmark$

Результат

$\text{sym}(p) := J(x, \lambda y p. (y = x), \text{refl}_x, y, p)$

Транзитивность из J: подробный вывод

Цель

Построить $\text{trans} : \prod_{x,y,z:A} (x =_A y) \rightarrow (y =_A z) \rightarrow (x =_A z)$

Стратегия: два применения J

Вариант 1: Фиксируем $p : x = y$, индуцируем по $q : y = z$

Мотив: $P(z, q) := (x =_A z)$

База: $P(y, \text{refl}_y) = (x =_A y)$ — обитаем p

Вариант 2: Индуцируем по $p : x = y$, фиксируем q

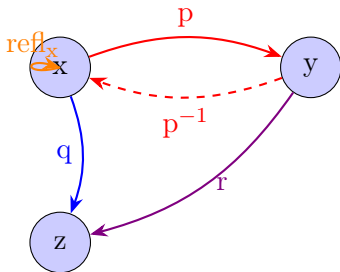
Мотив: $P(y, p) := (y = z) \rightarrow (x = z)$

База: $P(x, \text{refl}_x) = (x = z) \rightarrow (x = z)$ — обитаем $\lambda q. q$

Вычислительное поведение

$\text{trans}(\text{refl}_x, q) \equiv q$ и $\text{trans}(p, \text{refl}_y) \equiv p$ — зависит от выбора!

Группоид: интуиция через граф



Группоид — это граф, где:

- Вершины — элементы типа
- Рёбра — “пути” (доказательства равенства)
- Каждое ребро обратимо
- Пути можно соединять
- У каждой вершины есть петля (тождество)

Аналогия: города и дороги, где каждую дорогу можно проехать в обе стороны.

Аксиомы группоида для identity-типа

Что нужно проверить:

- ❶ Единица: $\text{trans}(\text{refl}, p) = p$ и $\text{trans}(p, \text{refl}) = p$
- ❷ Ассоциативность: $\text{trans}(\text{trans}(p, q), r) = \text{trans}(p, \text{trans}(q, r))$
- ❸ Обратный: $\text{trans}(\text{sym}(p), p) = \text{refl}$ и $\text{trans}(p, \text{sym}(p)) = \text{refl}$

Важно

Эти равенства — пропозициональные, не дефиниторные! Доказываются через J.

Типы как группоиды: соответствие

Группоид	Identity-тип	Обозначение
Объекты	элементы $x, y, z : A$	—
Морфизмы $x \rightarrow y$	пути $p : x =_A y$	$p : x \rightsquigarrow y$
Тождество id_x	рефлексивность	refl_x
Обратный p^{-1}	симметрия	$p^{-1} := \text{sym}(p)$
Композиция $q \circ p$	транзитивность	$p \cdot q := \text{trans}(p, q)$

Открытие Хофманна–Стрейхера (1994)

Не все типы — множества! В группоидной модели MLTT существует тип с двумя точками, у которого типы равенства не являются пропозициями (есть нетривиальные пути) $\Rightarrow$ UIP не выводим в чистой MLTT.

Высшие пути: от группоидов к ∞ -группоидам

Иерархия путей

- 0-пути: элементы $x : A$
- 1-пути: равенства $p : x =_A y$
- 2-пути: равенства между равенствами $\alpha : p =_{x=y} q$
- 3-пути: $\beta : \alpha =_{p=q} \gamma$
- И так далее...

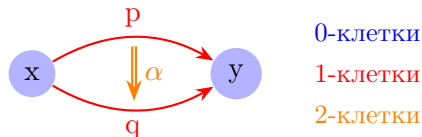
Пример 2-пути

Пусть $p, q : x = x$ — два цикла.
Тогда $\alpha : p = q$ — “гомотопия” между циклами.
В HoTT: α может быть нетривиальна!

∞ -группоид

Тип A с операциями на всех уровнях, согласованными “до когерентности”.

∞ -группоиды: бесконечная башня равенств



И так далее: 3-клетки между 2-клетками,
4-клетки между 3-клетками...

Идея ∞ -группоида:

- Не только точки и пути
- Но и “деформации путей”
- И “деформации деформаций”
- На всех уровнях!

Топологическая аналогия:

- Точки = элементы
- Пути = непрерывные кривые
- 2-пути = гомотопии (деформации)
- Пространство S^2 (сфера) имеет нетривиальные 2-пути!

∞ -категории: зачем это нужно?

Типы как ∞ -группоиды (интуиция)

Из identity-типов: элементы $\rightarrow$ пути $\rightarrow$ пути между путями $\rightarrow \dots$ даёт структуру ∞ -группоида. Это метатеоретический результат (Lumsdaine, van den Berg). Homotopy hypothesis Гротендика формулируется на уровне моделей.

Следствия:

- UIP “убивает” высшие уровни
- HoTT использует структуру
- UA: $(A \simeq B) = (A = B)$

Применения:

- Алгебраическая топология
- Высшая категорная логика
- Формализация математики

Для практики

В “обычном” Coq с UIP это неважно. Понимание помогает выбирать между Coq и HoTT.

h-уровни: классификация типов по сложности

Определение (h-уровни: индуктивное определение)

Определяем $\text{is-n-type}(A)$ индуктивно по $n \geq -2$:

База: $\text{isContr}(A) := \sum_{c:A} \prod_{x:A} \text{Id}_A(c, x)$ (h-level -2)

Шаг: $\text{is-}(n+1)\text{-type}(A) := \prod_{x,y:A} \text{is-n-type}(\text{Id}_A(x, y))$

Стандартные названия:

- $\text{isProp}(A)$: h-level -1 , $\text{isSet}(A)$: h-level 0 , $\text{isGroupoid}(A)$: h-level 1

Прим.: $\text{isProp}(A) \Leftrightarrow \prod_{x,y} x = y$ — эквивалентно, но не буквальное раскрытие.

Примеры h-уровней

h-level -2: КОНТРАКТИВНЫЕ

1, синглетон $\Sigma_{x:A} (a = x)$

h-level -1: пропозиции

0, логические утверждения

h-level 0: множества

$\mathbb{N}$, Bool — decidable eq

h-level ≥ 1

Type в HoTT — не множество!

UIP: Uniqueness of Identity Proofs

Определение (UIP — Uniqueness of Identity Proofs)

$$\text{UIP}(A) := \prod_{x,y:A} \prod_{p,q:\text{Id}_A(x,y)} \text{Id}_{\text{Id}_A(x,y)}(p,q)$$

Теорема (Эквивалентность формулировок)

Следующие утверждения эквивалентны:

- ❶ $\text{UIP}(A): \prod_{x,y:A} \prod_{p,q:x=y} p = q$
- ❷ $K(A): \prod_{x:A} \prod_{p:x=x} p = \text{refl}_x$
- ❸ $\text{isSet}(A): \prod_{x,y:A} \text{isProp}(\text{Id}_A(x,y))$

Теорема (Hedberg, 1998)

Пусть $\text{Dec}(A) := \prod_{x,y:A} (x = y) + \neg(x = y)$. Тогда $\text{Dec}(A) \rightarrow \text{UIP}(A)$.

Вопрос

Почему в MLTT может быть два разных доказательства $p, q : x = y$?

Аналогия: пути на Земле

Из Москвы в Токио можно долететь:

- через Европу (на запад)
- через Сибирь (на восток)

Оба пути соединяют те же точки, но они топологически различны — один нельзя непрерывно деформировать в другой.

Идея Хоффманна–Стрейхера: построить модель, где типы — не множества, а группоиды (объекты со структурой путей).

В теории типов

Тип `Bool` можно рассматривать как два города.

“Путь” $\sigma : \text{Bool} = \text{Bool}$ — это перестановка $\text{true} \leftrightarrow \text{false}$.

Есть два разных пути: тождественный и перестановка!

Независимость UIP: модель Хофманна–Стрейхера

Теорема (Hofmann–Streicher, 1994)

$\not\models_{\text{MLTT}} \Pi_{A:\mathcal{U}} \text{UIP}(A) \rightarrow \text{UIP}$ не выводим в чистой MLTT.

Доказательство (схема)

- 1 Интерпретируем типы как группоиды: $\llbracket A \rrbracket \in \mathbf{Gpd}$
- 2 Identity-тип $\Rightarrow$ морфизмы: $\llbracket x =_A y \rrbracket = \text{Hom}(x, y)$
- 3 Рассмотрим $\mathbf{Bool}$ как группоид с двумя объектами и автоморфизмом $\sigma : 0 \leftrightarrow 1$
- 4 Тогда $\sigma, \text{id} : \llbracket \mathbf{Bool} = \mathbf{Bool} \rrbracket$ — два разных морфизма
- 5 Значит, $\llbracket \text{UIP}(\mathbf{Bool}) \rrbracket$ ложно в этой модели

Следствие

$\text{MLTT} + \neg \text{UIP}$ непротиворечива $\Rightarrow$ основа для HoTT.

Ключ: если бы UIP был теоремой, он был бы истинен в любой модели.

Позиция Coq

В ядре Coq UIP не принимается по умолчанию — оставляет дверь для HoTT-библиотек.

Когда UIP выводим:

```
Require Import Eqdep_dec.  
Lemma UIP_nat :  $\forall (n:\text{nat})$   
  ( $p\ q : n = \bar{n}$ ),  $p = q$ .  
Proof. apply UIP_dec, Nat.eq_dec. Qed.
```

Аксиома K в Coq:

```
Require Import Eqdep.  
(* eq_rect_eq equivalent to K *)
```

Модули Eqdep, EqdepFacts содержат аксиому K и её следствия.

Coq vs HoTT: выбор аксиом

“Классический” Coq

- UIP/K выводимы для типов с decidable eq (Hedberg)
- Типы фактически трактуются как h-sets
- Proof irrelevance не в ядре, но стирается при extraction

Философия: доказательства — “спецификации”.

HoTT

- Отказ от UIP
- Типы $\sim$ пространства, пути $\sim$ гомотопии
- UA: $ua : (A \simeq B) \rightarrow (A = B)$ — изоморфизм

Философия: равенства вычислимы.

Практический вывод

Для большинства верификаций достаточно “сеточного” уровня — UIP не мешает.

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты
- 3 Identity-типы и равенство
- 4 Продвинутое темы: аксиомы и альтернативы**
- 5 CIC: Prop, Set, Type и импредикативность
- 6 Зависимые типы на практике
- 7 Заключение

Function Extensionality (funext)

Определение (Функциональная экстенциональность)

$$\text{funext} : (\prod_{x:A} f(x) = g(x)) \rightarrow (f = g)$$

Ключевой факт

Funext не выводима в чистой MLTT! Это независимое утверждение.

Почему важно?

- Без funext: $\lambda x. x + 0$ и $\lambda x. x$ не равны (разные термы)
- С funext: можно доказать равенство через поточечное
- В Coq: аксиома `Coq.Logic.FunctionalExtensionality`

Propositional Truncation $\|A\|$

Определение (Truncation)

Для типа A определяем $\|A\| : \text{Prop}$ с конструктором $|-| : A \rightarrow \|A\|$ и свойством $\text{isProp}(\|A\|)$.

Семантика

- $\|A\|$ обитаем $\Leftrightarrow A$ обитаем, но нельзя извлечь конкретный элемент
- Логический $\exists$: $\exists x. P(x) := \|\Sigma_{x:A} P(x)\|$

В Coq (интуиция)

Prop похож на truncation: стирается при extraction, элиминация в Type ограничена. Но это лишь аналогия — proof irrelevance не доказуем внутри Coq без аксиом.

Canonicity и проблема аксиом

Определение (Canonicity)

$\mathcal{T}$ обладает каноничностью: $\vdash_{\mathcal{T}} t : \mathbb{N} \implies \exists n. t \equiv S^n(\text{zero})$

Проблема: “застрявшие” термы

Аксиомы добавляют константы без вычислительных правил.

Пример: funext ломает вычисления

Пусть $f = \lambda x. x + 0$ и $g = \lambda x. x$. Поточечно равны, но $f \not\equiv g$.

С аксиомой funext можно построить:

$$p : f = g \quad \text{где} \quad p := \text{funext}(\lambda x. \text{plus_n_0}(x))$$

Теперь рассмотрим transport по p:

$$\text{transport}_P(p, v) : P(g) \quad \text{где} \quad v : P(f)$$

Проблема: transport не может вычислиться — p содержит funext, который “застрял”!

Почему это важно?

Если терм типа $\mathbb{N}$ не сводится к числу, мы не можем его вычислить — теряется computational content.

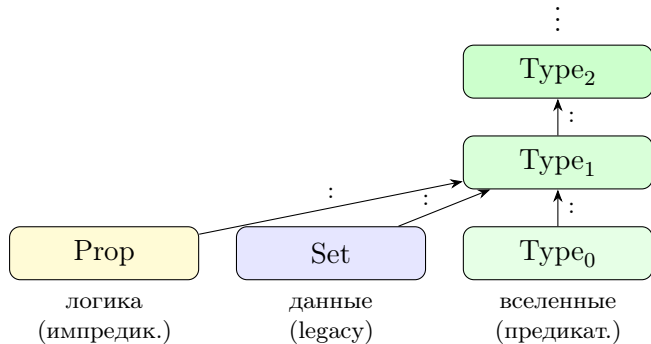
Решения

- Cubical Type Theory: равенства через интервал $\mathbb{I}$, каноничность доказана
Ключ: `funext` и `ua` получают вычислительные правила
- Setoids: работать с парой (тип, отношение $\sim$) вместо equality
Минус: много boilerplate, нужно доказывать уважение $\sim$
- Quotient types: факторизация по $\sim$ как primitive (Lean, частично Coq)

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты
- 3 Identity-типы и равенство
- 4 Продвинутые темы: аксиомы и альтернативы
- 5 CIC: Prop, Set, Type и импредикативность**
- 6 Зависимые типы на практике
- 7 Заключение

Сорта в Coq: иерархия вселенных



- $\text{Prop} : \text{Type}_1$ — мир логических утверждений (импредикативный)
- $\text{Set} : \text{Type}_1$ — “мир данных”, формально $\approx \text{Type}_0$, но со своими опциями (impredicative Set, extraction)
- $\text{Type}_0 : \text{Type}_1 : \text{Type}_2 : \dots$ — кумулятивная иерархия

Импредикативность vs предикативность

Определение (Импредикативность)

$$\frac{\Gamma \vdash A : s \quad \Gamma, x : A \vdash B : \text{Prop}}{\Gamma \vdash \Pi_{x:A} B : \text{Prop}}$$

где $s \in \{\text{Prop}, \text{Set}, \text{Type}_i\}$.

Результат в Prop , даже если $A : \text{Type}_i$!

Предикативные Type_i

$\forall X : \text{Type}_i, \dots : \text{Type}_{i+1}$

Пример: $\forall X : \text{Type}_0, X \rightarrow X : \text{Type}_1$

Квантификация поднимает уровень.

Почему это важно?

Импредикативный $\text{Type} + \text{Type} : \text{Type} \Rightarrow$ парадокс Жирара!

CIC: конкретизация иерархии вселенных

CIC конкретизирует эту идею

- Prop выделен как особый сорт — импредикативный, но “безопасный”
- Type_i — предикативная лестница для данных
- Кумулятивность: если $A : \text{Type}_i$, то $A : \text{Type}_{i+1}$

Практика: Coq автоматически выводит уровни вселенных (universe inference).

Импредикативные кодировки в Prop

Church-кодировка конъюнкции

Definition `and_imp (A B : Prop) : Prop :=`
`∀ P : Prop, (A → B → P) → P.`

`forall P : Prop` — импредикативная квантификация (можно подставлять `and_imp`).

Аналогичные кодировки

Definition `or_imp (A B:Prop) : Prop := ∀ P:Prop, (A→P) → (B→P) → P.`

Definition `ex_imp (A:Type) (Q:A→Prop) : Prop := ∀ P:Prop, (∀ x, Q x → P) → P.`

Напоминание: Church-кодировки в System F

$\text{Pair } \alpha \beta \equiv \forall \gamma. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma, \quad \text{List } \alpha \equiv \forall \beta. (\alpha \rightarrow \beta \rightarrow \beta) \rightarrow \beta \rightarrow \beta$

Разница в Coq

- System F: импредикативность для всех типов
- CIC: импредикативность только для Prop; “данные” предикативны

Следствие

Church-кодировку List можно в Type, но на уровень выше. Для данных — индуктивные типы.

Extraction: Prop стирается

Ключевое свойство Prop

Всё, что живёт в Prop, стирается при extraction в OCaml/Haskell.

Coq:

```
Definition safe_div
  (n m : nat)
  (pf : m <> 0) : nat :=
  (* ... implementation ... *)
  n / m.
```

Extracted OCaml:

```
let safe_div n m =
  (* pf erased! *)
  n / m
```

- Доказательство $pf : m \neq 0$ живёт в Prop
- При extraction оно исчезает — zero runtime cost
- Всё, что в Type/Set — реальные данные и функции

Вывод: Prop — “спецификации”, Type — “реализации”.

Индуктивные типы и позитивность

Определение (Строгая позитивность)

Тип I строго позитивен в типе T , если I встречается в T только:

- 1 как сам $T = I$, или
- 2 справа от $\rightarrow$ в $T = A \rightarrow T'$, где $I \notin \text{FV}(A)$ и I строго позитивен в T'

Корректно:

```
Inductive list (A : Type) :=  
| nil : list A  
| cons : A → list A → list A.
```

Запрещено:

```
(* DOES NOT COMPILE! *)  
Inductive bad :=  
| C : (bad → nat) → bad.
```

bad слева от $\rightarrow$ — парадокс!

Правило

Тип I может появляться только в строго позитивных позициях: как результат, но не слева от $\rightarrow$.

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты
- 3 Identity-типы и равенство
- 4 Продвинутое темы: аксиомы и альтернативы
- 5 CIC: Prop, Set, Type и импредикативность
- 6 Зависимые типы на практике
- 7 Заключение

Пример: $\text{Vec } A \ n$ — вектор фиксированной длины

```
Inductive vec (A : Type) : nat → Type :=  
| vnil : vec A 0  
| vcons : ∀ n, A → vec A n → vec A (S n).
```

Что это даёт?

Длина вектора — часть типа; $\text{vec nat } 3$ и $\text{vec nat } 5$ — разные типы; операции с несовместимыми длинами — ошибка компиляции.

Безопасный head (упрощённый псевдокод)

```
Definition vhead (A:Type) (n:nat) (v:vec A (S n)) : A :=  
  match v in vec _ k return match k with 0 ⇒ unit | S _ ⇒ A end  
  with vnil ⇒ tt | vcons _ x _ ⇒ x end.
```

Тип $\text{vec } A \ (S \ n)$ гарантирует непустоту. В реальном Coq нужен зависимый match с `in...return`.

Dependent pattern matching

Почему `vnil` не рассматривается?

Аргумент имеет тип `vec A (S n)`, а `vnil : vec A 0`. Индексы `S n` и `0` различны — `Coq` автоматически исключает невозможные случаи.

Конкатенация с точными размерами (упрощённо)

```
Fixpoint vappend (A:Type) (n m:nat) (v1:vec A n) (v2:vec A m)
  : vec A (n + m) := match v1 in vec _ n' return vec A (n' + m) with
| vnil => v2 | vcons k x xs => vcons (k+m) x (vappend A k m xs v2) end.
```

Тип результата `vec A (n + m)` — точная сумма длин! Требуется `in...return` для зависимой типизации.

Тензоры как зависимые типы

`vec` — простейший проху для тензоров: строка изображения (`vec Pixel 1920`), батч (`vec Sample 32`), embedding (`vec Float 768`).

Гарантии на уровне типов

- Несоответствие размеров — ошибка компиляции, не runtime crash
- Shape mismatch обнаруживается статически

Библиотеки: Idris, Agda, dependent Haskell (singletons), dex-lang.

ML-инвариант: Train vs Test

Проблема: data leakage

Случайное использование тестовых данных при обучении — частая ошибка.

Inductive split := Train | Test.

Inductive dataset (A : Type) : split → Type :=

| train_ds : list A → dataset A Train

| test_ds : list A → dataset A Test.

Definition model (A : Type) := A → bool.

Parameter train_model : ∀ A, dataset A Train → model A.

Что это даёт?

Тип train_model физически не позволяет передать test_ds!

План лекции

- 1 Введение: от MLTT к Coq
- 2 MLTT: синтаксис и варианты
- 3 Identity-типы и равенство
- 4 Продвинутые темы: аксиомы и альтернативы
- 5 CIC: Prop, Set, Type и импредикативность
- 6 Зависимые типы на практике
- 7 Заключение**

Новое в лекции 10:

- Варианты MLTT (ITT/ETT)
- Identity-типы и J подробно
- Симметрия/транзитивность из J
- Группоиды и h-уровни
- UIP и Хофманн–Стрейхер
- Ландшафт proof assistants

Из лекции 10 (Coq):

- CIC: Prop/Set/Type_i
- Импредикативный Prop
- Extraction
- Positivity check
- vec, ML-инварианты

- ❶ ITT vs ETT: Coq — интенциональная, проверка типов разрешима
- ❷ Identity-тип и элиминатор J дают все свойства равенства
- ❸ Типы — группоиды: sym , trans , аксиомы группоида доказуемы
- ❹ h -уровни: контрактивные $\rightarrow$ пропозиции $\rightarrow$ множества $\rightarrow \dots$
- ❺ UIP не выводим в чистой MLTT (Хофманн–Стрейхер)
- ❻ Coq : UIP опционален, Prop импредикативен и стирается

Литература и ресурсы

Основные источники

- Software Foundations (Benjamin Pierce et al.) — практика Coq
- CPDT (Adam Chlipala) — продвинутые техники
- HoTT Book — для интересующихся HoTT

Документация

- Coq Reference Manual: <https://coq.inria.fr/refman/>
- Coq'Art (Bertot, Castéran) — классический учебник

Для ML + формальные методы

- MLCoq, Coquelicot (анализ в Coq)
- Verified Software Toolchain (VST)
- Papers: “Verified Robustness”, “Formal Verification of Neural Networks”

Спасибо за внимание!

Вопросы?